Parallel & Distributed Computing - Milestone 2
Distributed Web Crawler & Page Rank

Page Rank Algorithm Comparison

Sequential – Centralised Aggregation – Distributed Reduction

Hamza Ahmed - 29265
Muhammad Hamza Arif - 29234
Laraib Fatima Farooqui - 26521
Arbaaz Murtaza - 29052

Overview:

This document benchmarks the three explored PageRank strategies on a real web-crawl graph. It builds on the web-crawl graphs produced in Milestone 1, implementing and benchmarking three PageRank strategies across three graph scales (Depth 1, 2, and 3 crawls) to evaluate the impact of graph size on parallel performance.

Field	Details
Hardware	AMD Ryzen 5 7430U · 8 GB DDR4 · 256 GB SSD
CPU Cores	6 physical / 12 logical · Best Performance mode
Ray Workers	12 (one per logical core)
Framework	Ray 2.54.0 + NumPy 2.4.3 + Pandas 3.0.1

Damping Factor	0.85
Convergence Tolerance	1e-6 (L∞ norm)
Fixed Iterations	25
Runs	Run 1 (Depth 1) · Run 2 (Depth 2) · Run 3 (Depth 3)

Algorithms Compared:

Algorithm	Description	Parallelism
Sequential	Single-process iterative PageRank using NumPy. Baseline for all speedup calculations.	None (1 process)
Centralized Aggregation	Workers compute chunks; the driver collects all results each iteration via ray.get() and writes back the updated rank vector. Communication cost: O(num_workers) per iteration.	12 Ray workers
Distributed Reduction	Workers compute chunks; results are merged via a binary tree of Ray tasks. Only 1 ray.get() per iteration. Communication cost: O(log num_workers) per iteration.	12 Ray workers

Termination Policy:

Every algorithm is evaluated under two stopping conditions:

- ***Fixed Iteration Count:*** always runs exactly 25 iterations regardless of convergence.
- ***Convergence-Based:*** stops when the L∞ norm of rank change drops below the tolerance of 1e-6.

Python Dictionary Overhead VS Compressed Sparse Row (CSR):

The graph is loaded from JSON and immediately converted to a NumPy CSR structure. This choice is critical for memory efficiency and performance at scale.

Why Native Python Structures Are Inefficient

- **Hash Table Sizing:** Python dictionaries allocate sparse arrays to ensure O(1) lookups, typically operating at only 66% capacity to avoid hash collisions.
- **Object Boxing:** In C or NumPy, a 32-bit integer requires exactly 4 bytes. Standard Python wraps integers in a PyLongObject (28 bytes) and strings in a PyUnicodeObject (50+ bytes just for the header).
- **List Pointers:** Python lists do not store contiguous data. They store 8-byte memory pointers that reference scattered objects across RAM, destroying cache locality.

Memory Comparison for a 500 MB JSON Graph

Assuming a 500 MB graph contains several million nodes and 15–20 million edges:

Structure	Estimated RAM Usage	Reason
Python Dict of Lists	2.5 GB – 5.0 GB	PyDictObject structures, 8-byte edge pointers, boxed string/integer objects
NumPy CSR (int32)	~80 MB – 150 MB	Three flat, contiguous C-arrays — eliminates pointer overhead and object boxing entirely

The CSR structure reduces topology to three NumPy arrays: indptr (slice boundaries), indices (inbound neighbour list), and out_counts (outbound degree per node). Once built, the raw Python dict is freed from memory. This is why the CSR build step is measured separately and reported for each run.

Observed CSR Build Times

Run	Nodes	CSR Build Time
Run 1 (Depth 1)	54,367	0.15s
Run 2 (Depth 2)	323,223	1.51s
Run 3 (Depth 3)	1,502,297	7.77s

Code Architecture:

Shared Utilities

All three algorithms share a common data preparation pipeline:

- *load_graph(graph_file)*: Reads crawl_graph.json into a Python dict.
- *build_index_maps(graph)*: Assigns each unique URL a stable integer index, producing node_to_idx and idx_to_node mappings.
- *build_csr(graph, node_to_idx)*: Constructs three NumPy arrays: indptr (slice boundaries for inbound neighbours), indices (concatenated inbound neighbour lists), and out_counts (outbound link count per node). This is built once and reused by all algorithms.
- *prepare_shared_structures(graph_file)*: Orchestrates the above steps, frees the raw dict from memory after CSR is built, and prints timing and size statistics.

Sequential PageRank

Function: run_sequential_pagerank()

Standard iterative PageRank implemented entirely in NumPy within a single process. Key design choices:

- Uses a swap buffer (pagerank / new_ranks) to avoid allocating a new array each iteration.
- Dangling node mass is implicitly handled by the (1-d)/N term in the update formula.
- Supports both termination modes via the fixed_iters parameter: if set, runs exactly that many iterations; if None, runs until convergence.
- Records per-iteration time (iter_times) and max L∞ diff (iter_diffs) for later analysis.

Centralized Aggregation (Ray)

Function: run_centralized_aggregation() | Worker: _ca_compute_chunk()

Parallelises PageRank by splitting the node array into chunks, one per worker. Architecture:

- The CSR arrays (indptr, indices, out_counts) are pushed into the Ray object store once via ray.put() for zero-copy reads by workers.
- The current rank vector is written to a memory-mapped file (numpy.memmap) so workers can read it without serialisation overhead.
- For each iteration: workers compute new ranks for their chunk in parallel, the driver collects all results via ray.get() (communication step), then writes the merged vector back to the mmap file.
- Communication overhead is explicitly measured: the time spent inside ray.get() is recorded as comm_times each iteration.
- This architecture has O(num_workers) ray.get() calls per iteration, meaning that the driver is a serial bottleneck.

Distributed Reduction (Ray)

Function: run_distributed_reduction() | Workers: _dr_compute_chunk(), _reduce_pair()

Uses the same chunk-based compute workers as Centralized Aggregation, but merges results via a binary tree of Ray tasks instead of collecting in the driver:

- After workers produce their chunk results, _reduce_pair() tasks merge pairs of results within Ray, keeping intermediate arrays in the object store.
- Only 1 ray.get() per iteration (the merged root) which eliminates the O(num_workers) serialisation bottleneck of Centralized Aggregation.
- The tree depth is ceil(log2(num_workers+1)). For 4 workers, this is 3 rounds, producing 2 reduction rounds in practice.
- The trade-off is additional task-scheduling overhead from the extra Ray tasks in each reduction round.

Run 1 – Depth 1:

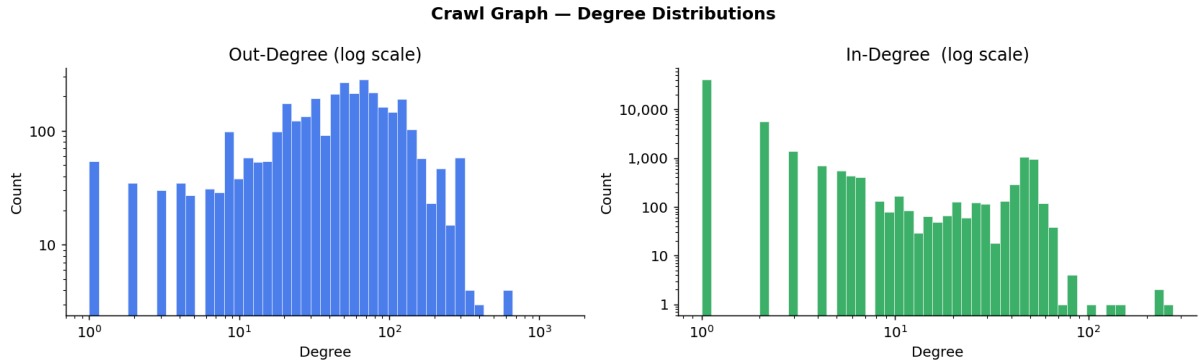
Graph Statistics:

Graph: crawl_graph_1.json

CSR build time: 0.15s

Property	Value
Total Nodes	54,367
Total Directed Edges	214,861
Source Nodes (raw graph)	3,355
Avg Out-Degree	3.95
Max Out-Degree	1,395
Avg In-Degree	3.95
Max In-Degree	274
Dangling Nodes (out=0)	51,012
Sink Nodes (in=0)	35
Graph Density	7.27e-05

Degree Distribution:



Results:

Benchmark Summary

Algorithm	Mode	Iters	Conv. At	Total Time (s)	Avg Iter (s)	Speedup vs Seq	Comm %	Final Max Diff
Sequential	Fixed	25	25	11.312	0.4524	1.000×	0.0%	1.02e-09
Sequential	Convergence	7	7	3.099	0.4427	1.000×	0.0%	6.16e-07
Centralized Aggregation	Fixed	25	25	7.313	0.2924	1.547×	51.2%	1.02e-09
Centralized Aggregation	Convergence	7	7	1.762	0.2516	1.759×	50.0%	6.16e-07
Distributed Reduction	Fixed	25	25	7.064	0.2825	1.601×	0.0%	1.02e-09

Distributed Reduction	Convergence	7	7	1.930	0.2756	1.606×	0.0%	6.16e-07

Per-Experiment Iteration Logs

Experiment 1: Sequential, Fixed (25 Iterations)

Iteration	Max Diff	Iter Time (s)
1	1.25e-04	0.557s
5	2.71e-06	0.463s
10	1.29e-07	0.433s
15	2.55e-08	0.452s
20	5.11e-09	0.453s
25	1.02e-09	0.430s

Experiment 2: Sequential, Convergence

Iteration	Max Diff	Iter Time (s)
1	1.25e-04	1.032
5	2.71e-06	0.396

7 (conv.)	< 1e-06	~0.396

Experiment 3: Centralized Aggregation, Fixed (25 iterations)

Iteration	Max Diff	Iter Time (s)	Comm Time (s)
1	1.25e-04	0.279s	0.137s
5	2.71e-06	0.283s	0.141s
10	1.29e-07	0.272s	0.129s
15	2.55e-08	0.292s	0.152s
20	5.11e-09	0.369s	0.183s
25	1.02e-09	0.263s	0.124s

Experiment 4: Centralized Aggregation, Convergence

Iteration	Max Diff	Iter Time (s)	Comm Time (s)
1	1.25e-04	0.264s	0.142s
5	2.71e-06	0.242s	0.120s
7 (conv.)	< 1e-06	—	—

Experiment 5: Distributed Reduction, Fixed (25 Iterations)

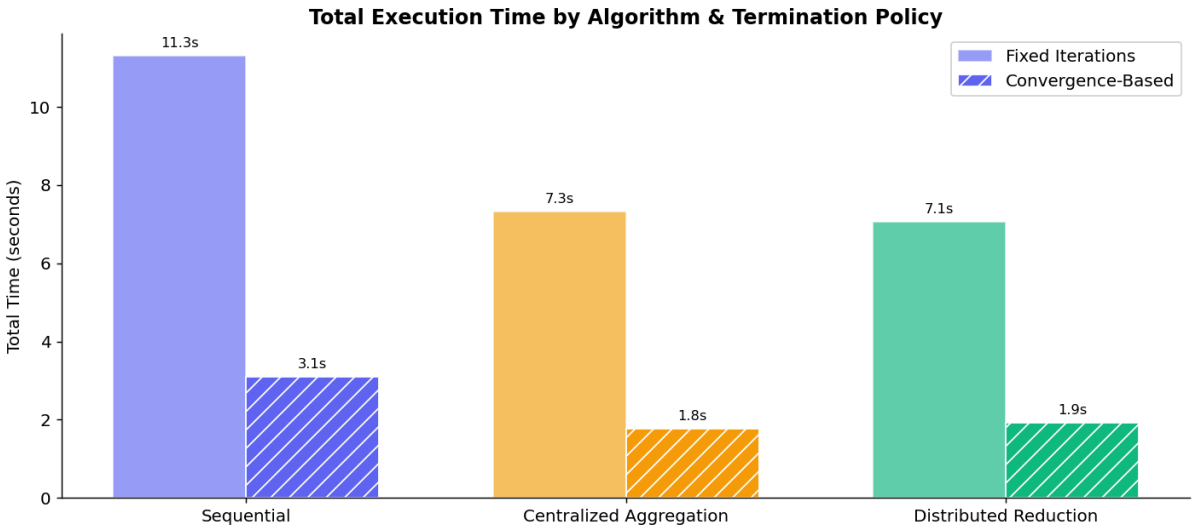
Iteration	Max Diff	Iter Time (s)	Reduce Rounds
1	1.25e-04	0.294s	4
5	2.71e-06	0.288s	4
10	1.29e-07	0.354s	4
15	2.55e-08	0.247s	4
20	5.11e-09	0.249s	4
25	1.02e-09	0.259s	4

Experiment 6: Distributed Reduction, Convergence

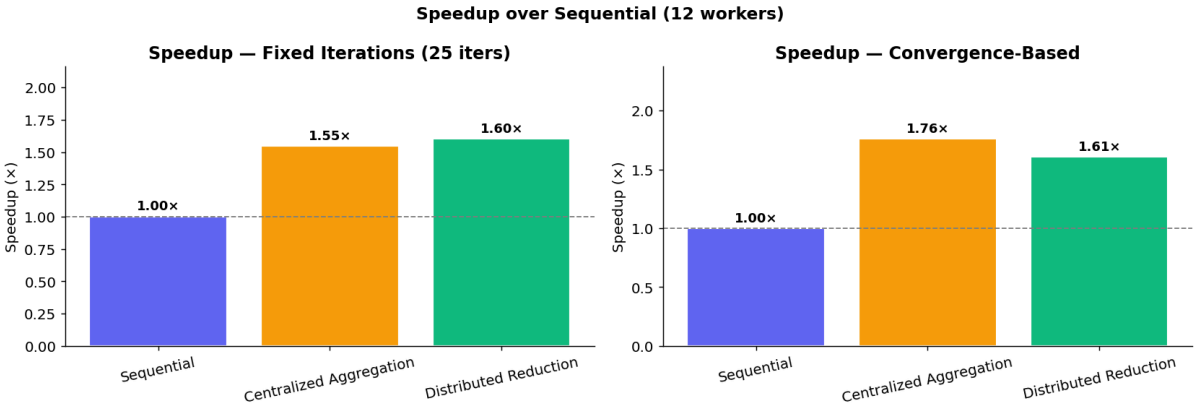
Iteration	Max Diff	Iter Time (s)	Reduce Rounds
1	1.25e-04	0.240s	4
5	2.71e-06	0.354s	4
7 (conv.)	< 1e-06	—	4

Performance Charts:

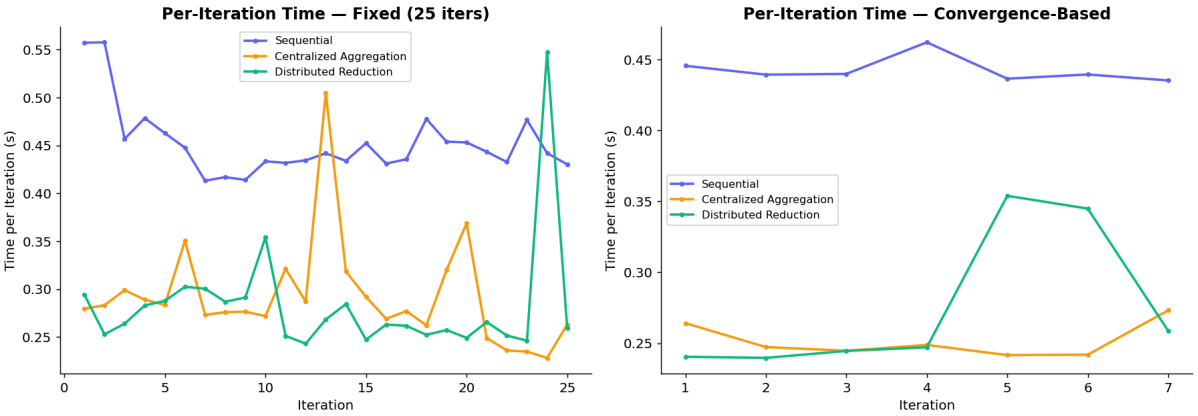
Total Execution Time



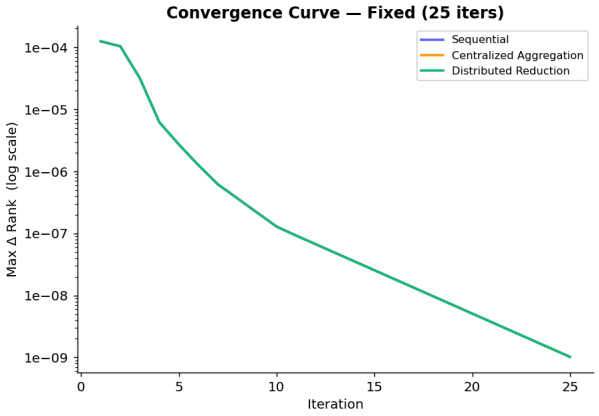
SpeedUp vs Sequential Baseline

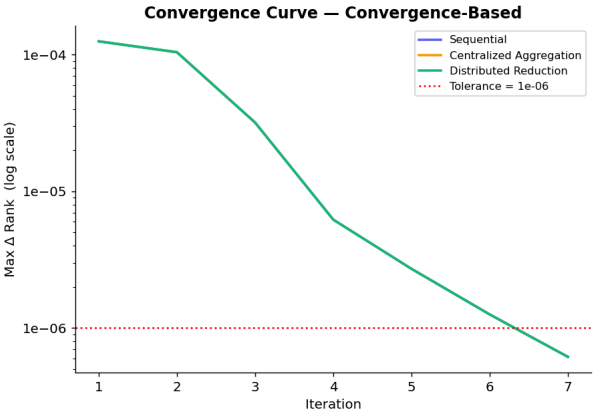


Per-Iteration Time Breakdown

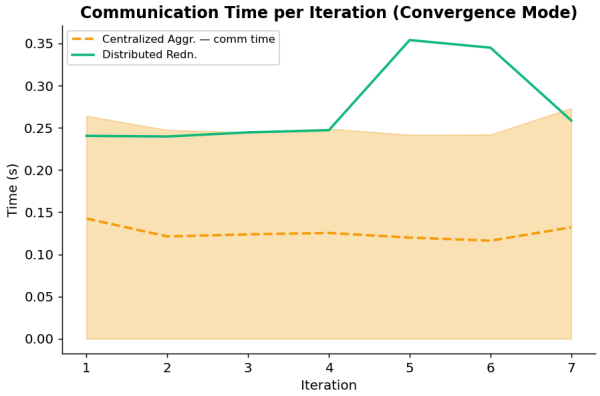


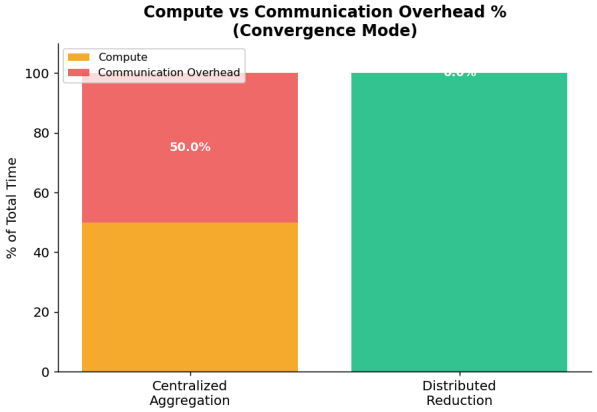
Convergence Curves



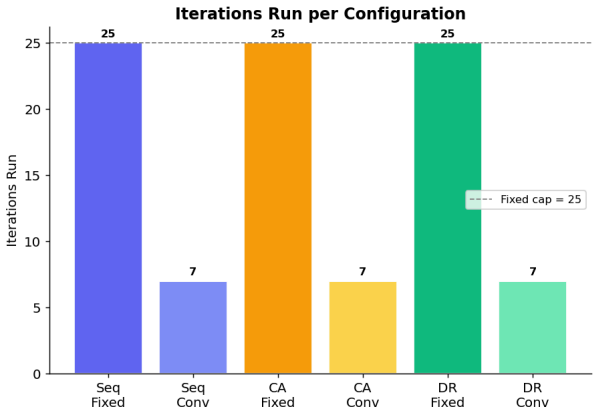


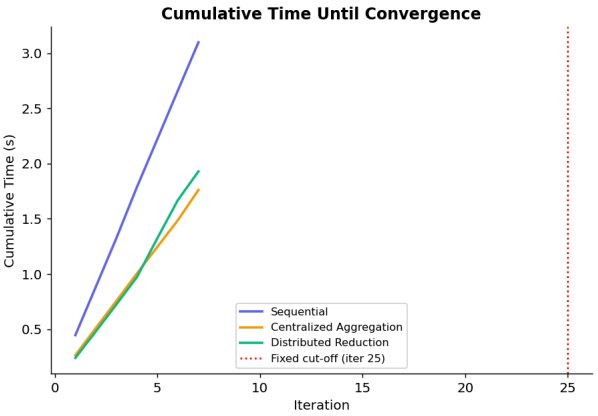
Communication Overhead



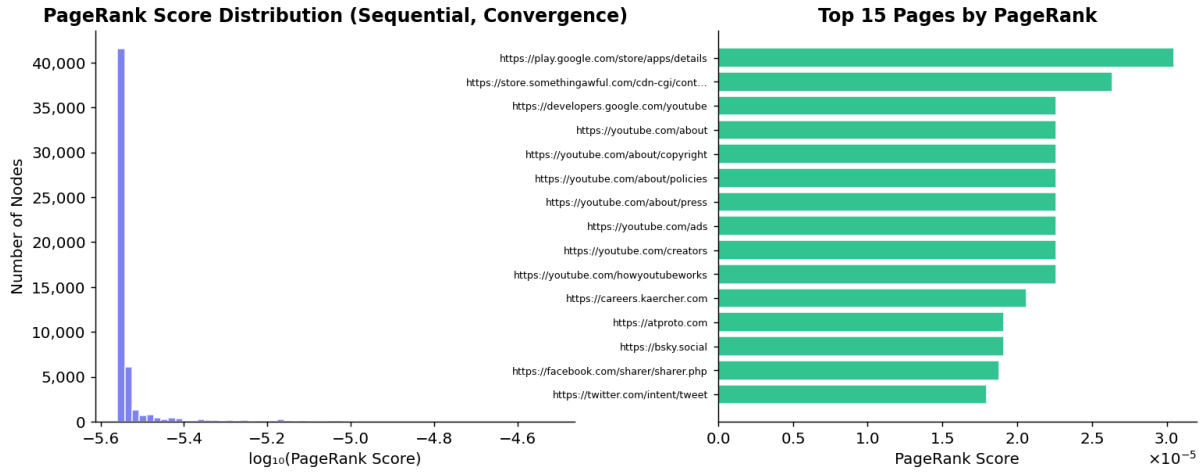


Fixed vs Convergence; Iteration Trade-Off

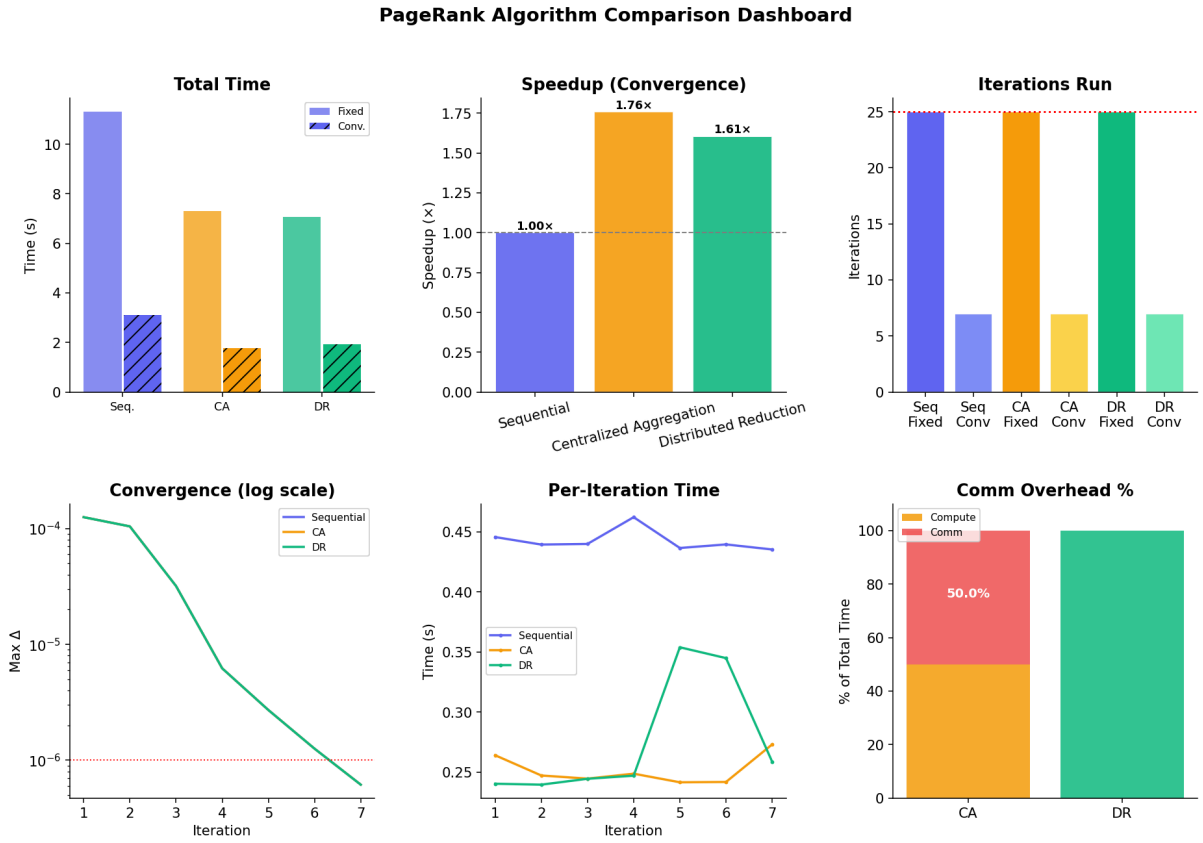




PageRank Score Distribution & Top Pages



Algorithm Comparison Dashboard



Top 10 Pages by PageRank

Rank	URL	Score
1	https://play.google.com/store/apps/details	0.00003048

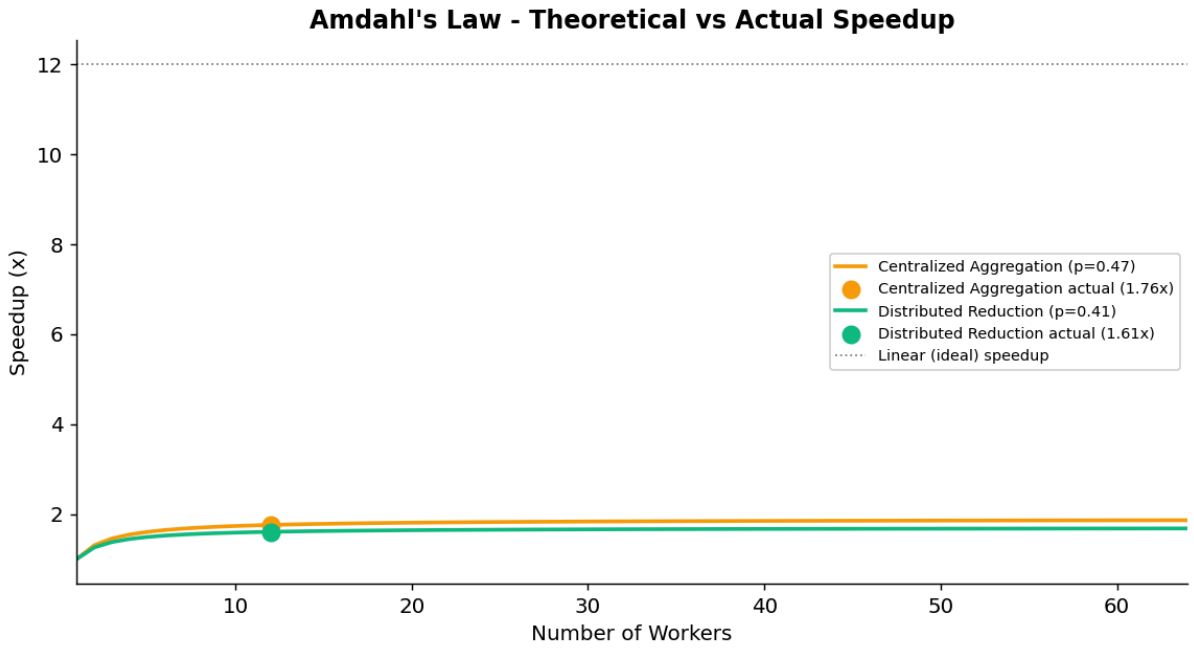
2	https://store.somethingawful.com/cdn-cgi/content	0.00002639
3	https://developers.google.com/youtube	0.00002260
4	https://youtube.com/about	0.00002260
5	https://youtube.com/about/copyright	0.00002260
6	https://youtube.com/about/policies	0.00002260
7	https://youtube.com/about/press	0.00002260
8	https://youtube.com/ads	0.00002260
9	https://youtube.com/creators	0.00002260
10	https://youtube.com/howyoutubeworks	0.00002260

Numerical Correctness

Comparison	Max Abs Diff	Pass (atol=1e-5)	Top-5 URLs Match
Sequential vs Centralized	0.00e+00	✓ PASS	✓ All 5
Sequential vs Distributed	0.00e+00	✓ PASS	✓ All 5
Centralized vs Distributed	0.00e+00	✓ PASS	✓ All 5

Amdahl's Law Analysis

Algorithm	Actual Speedup	Parallel Efficiency	Estimated p	Theoretical Max
Centralized Aggregation	1.759×	14.7%	0.471	1.9×
Distributed Reduction	1.606×	13.4%	0.411	1.7×



Fixed VS Convergence – Rank Quality

Algorithm	Max |fixed−conv|	Mean |fixed−conv|	Top-100 Overlap	Conv. Iter
Sequential	8.9769e-07	5.0201e-09	92%	7
Centralized Aggregation	8.9769e-07	5.0201e-09	92%	7
Distributed Reduction	8.9769e-07	5.0201e-09	92%	7

Key Conclusions

- CA achieves 1.759× speedup (convergence) and 1.547× (fixed)
- DR achieves 1.606× speedup (convergence) and 1.601× (fixed)
- CA comm overhead: 50.0% (convergence)
- Converged in 7 iterations
- At this small scale, CA edges out DR slightly in convergence mode, but DR's advantage is its zero driver-side comm overhead
- Top-100 URL overlap fixed vs convergence: 92%

Run 2 – Depth 2:

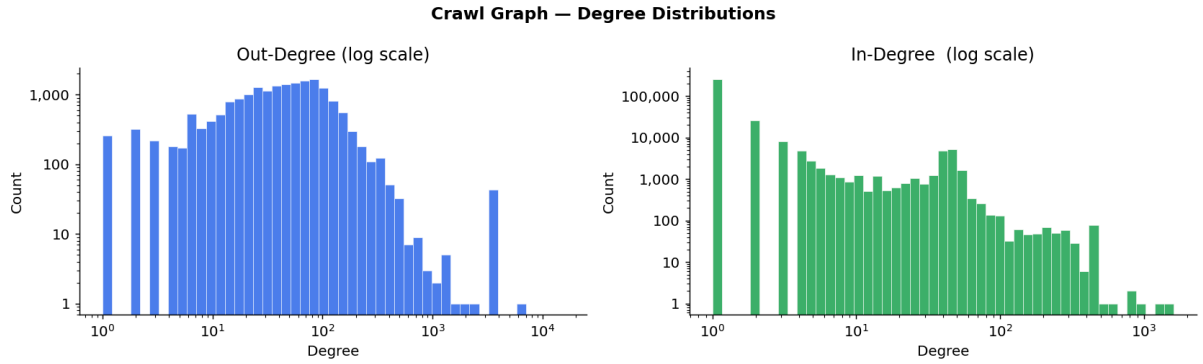
Graph Statistics:

Graph: crawl_graph_2.json

CSR build time: 1.51s

Property	Value
Total Nodes	323,223
Total Directed Edges	1,300,951
Source Nodes (raw graph)	18,888
Avg Out-Degree	4.02
Max Out-Degree	15,464
Avg In-Degree	4.02
Max In-Degree	1,612
Dangling Nodes (out=0)	304,335
Sink Nodes (in=0)	26
Graph Density	1.25e-05

Degree Distribution:



Results:

Benchmark Summary

Algorithm	Mode	Iters	Conv. At	Total Time (s)	Avg Iter (s)	Speedup vs Seq	Comm %	Final Max Diff
Sequential	Fixed	25	25	67.244	2.6869	1.000×	0.0%	2.63e-09
Sequential	Convergence	7	7	17.908	2.5582	1.000×	0.0%	6.94e-07
Centralized Aggregation	Fixed	25	25	30.975	1.2389	2.171×	68.8%	2.63e-09
Centralized Aggregation	Convergence	7	7	7.640	1.0913	2.344×	66.5%	6.94e-07
Distributed Reduction	Fixed	25	25	29.929	1.1971	2.247×	0.0%	2.63e-09
Distributed Reduction	Convergence	7	7	7.572	1.0816	2.365×	0.0%	6.94e-07

Per-Experiment Iteration Logs

Experiment 1: Sequential, Fixed (25 Iterations)

Iteration	Max Diff	Iter Time (s)
1	1.05e-04	3.772s
5	3.01e-06	2.672s
10	1.23e-07	2.997s
15	1.95e-08	2.604s
20	6.36e-09	2.570s
25	2.63e-09	2.625s

Experiment 2: Sequential, Convergence

Iteration	Max Diff	Iter Time (s)
1	1.05e-04	2.618s
5	3.01e-06	2.538s

Converged at itr=7

Experiment 3: Centralized Aggregation, Fixed (25 iterations)

Iteration	Max Diff	Iter Time (s)	Comm Time (s)
1	1.05e-04	5.046s	4.498s
5	3.01e-06	1.242s	0.832s
10	1.23e-07	0.925s	0.617s
15	1.95e-08	1.027s	0.662s
20	6.36e-09	0.922s	0.644s
25	2.63e-09	1.149s	0.776s

Experiment 4: Centralized Aggregation, Convergence

Iteration	Max Diff	Iter Time (s)	Comm Time (s)
1	1.05e-04	1.120s	0.709
5	3.01e-06	1.074s	0.706

Converged at itr=7

Experiment 5: Distributed Reduction, Fixed (25 Iterations)

Iteration	Max Diff	Iter Time (s)	Reduce Rounds
1	1.05e-04	1.153s	4

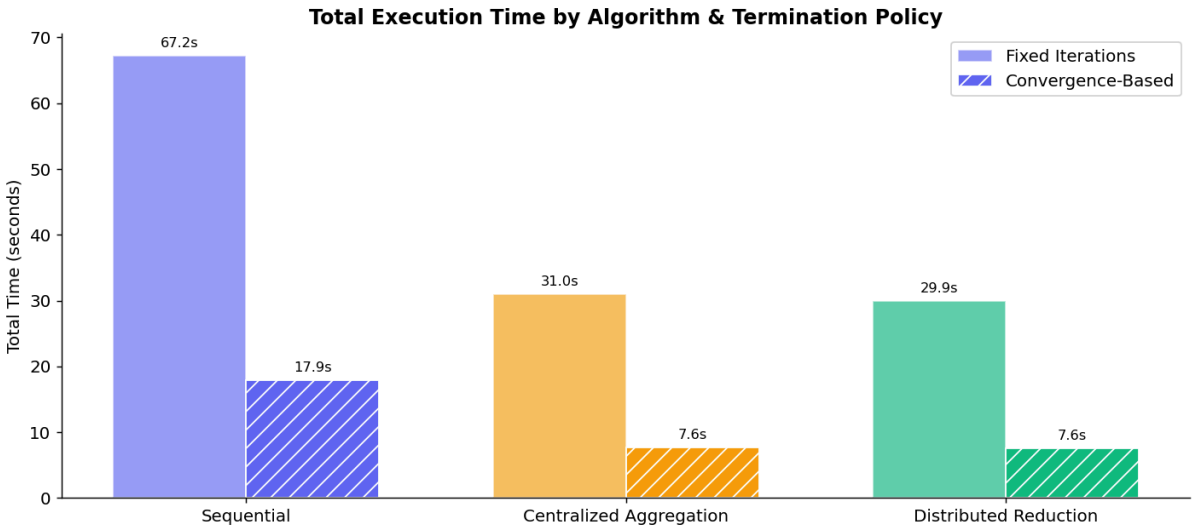
5	3.01e-06	1.073s	4
10	1.23e-07	1.071s	4
15	1.95e-08	1.094s	4
20	6.36e-09	0.954s	4
25	2.63e-09	0.999s	4

Experiment 6: Distributed Reduction, Convergence

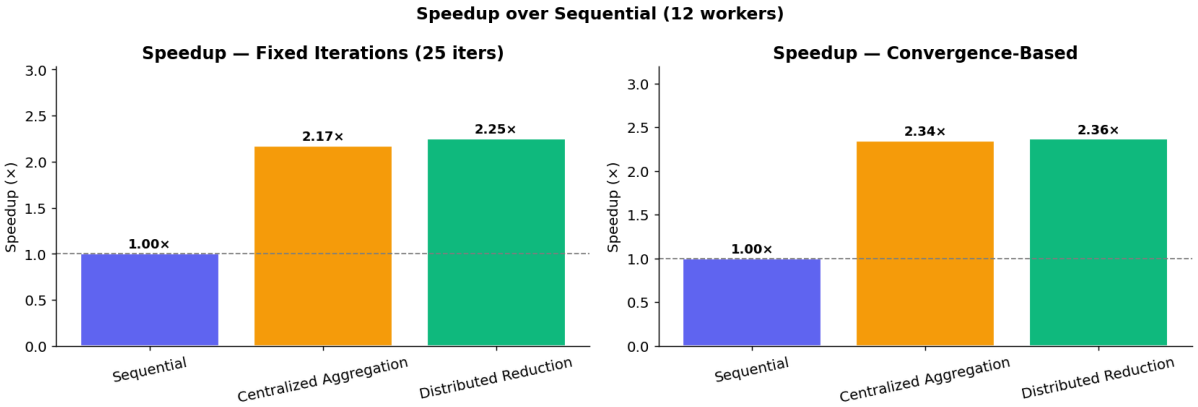
Iteration	Max Diff	Iter Time (s)	Reduce Rounds
1	1.05e-04	0.964s	4
5	3.01e-06	1.023s	4
7 (conv.)	< 1e-06	—	4

Performance Charts:

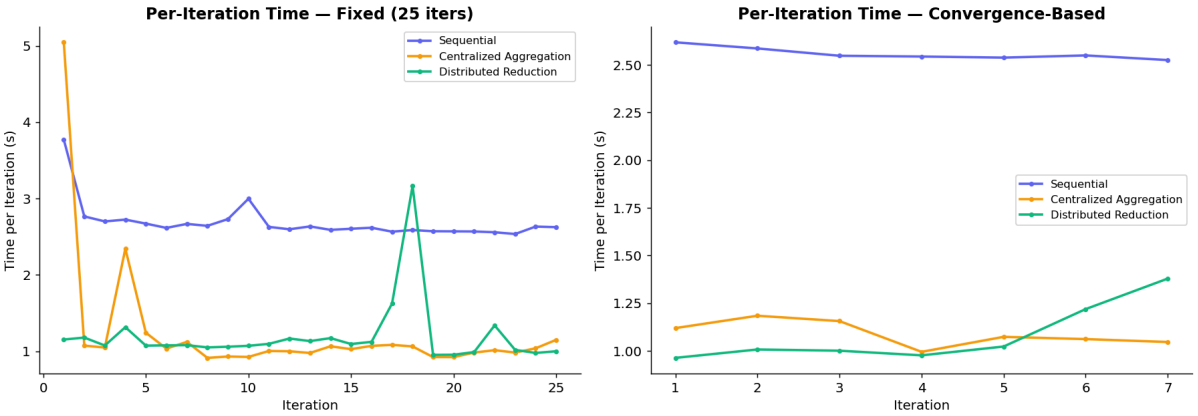
Total Execution Time



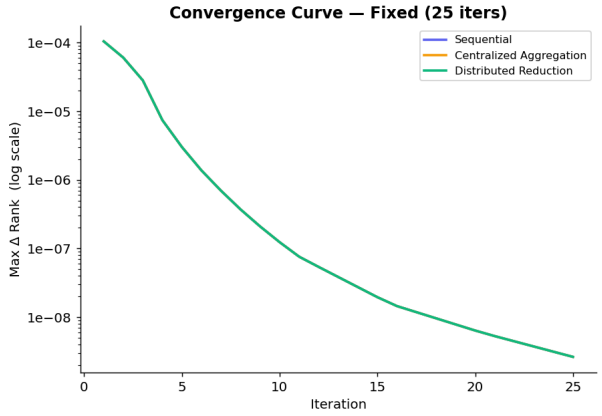
SpeedUp vs Sequential Baseline

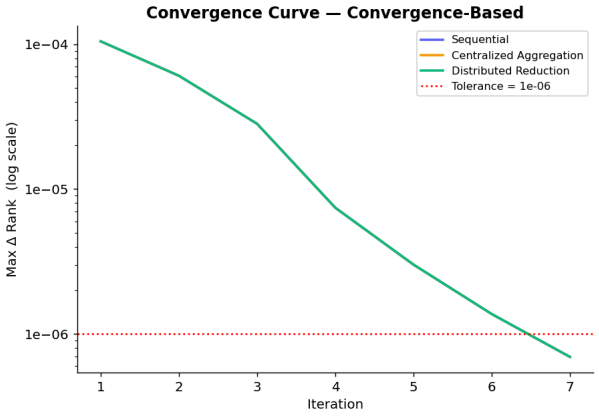


Per-Iteration Time Breakdown

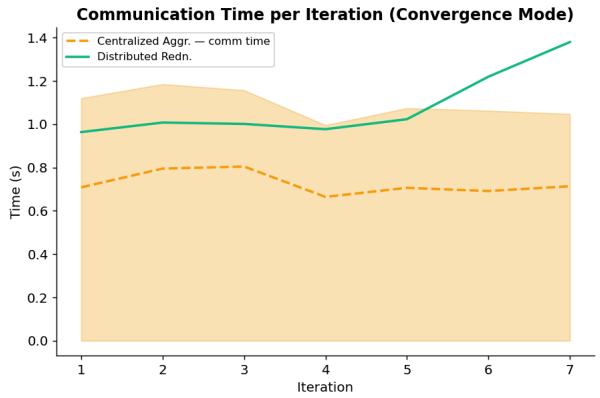


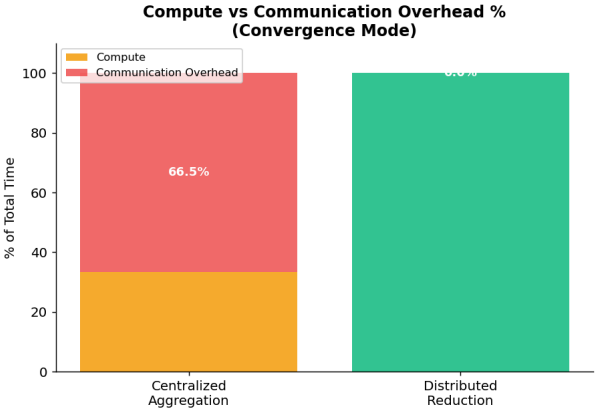
Convergence Curves



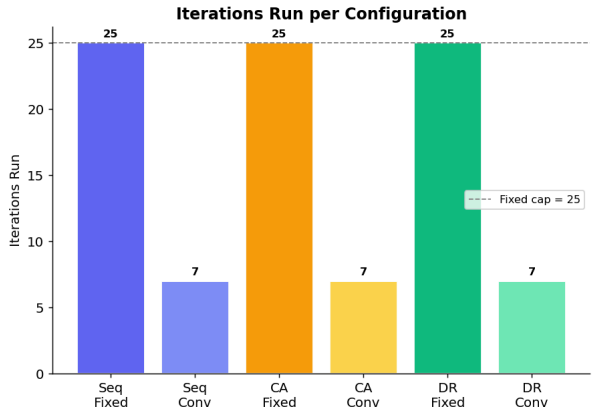


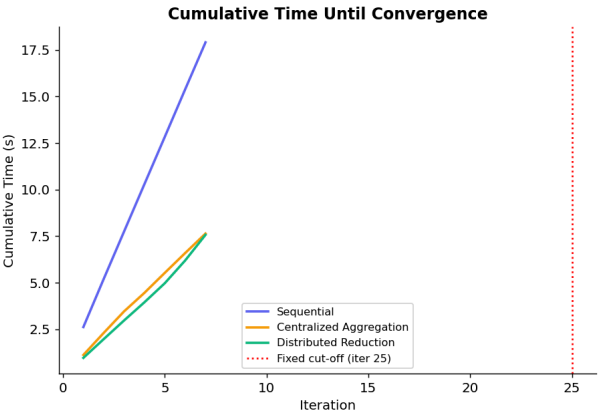
Communication Overhead



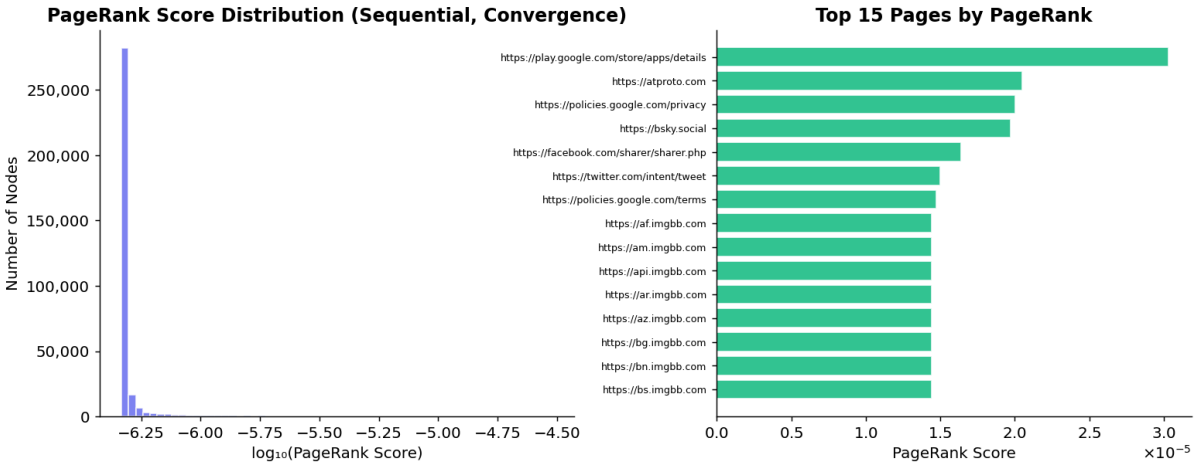


Fixed vs Convergence; Iteration Trade-Off

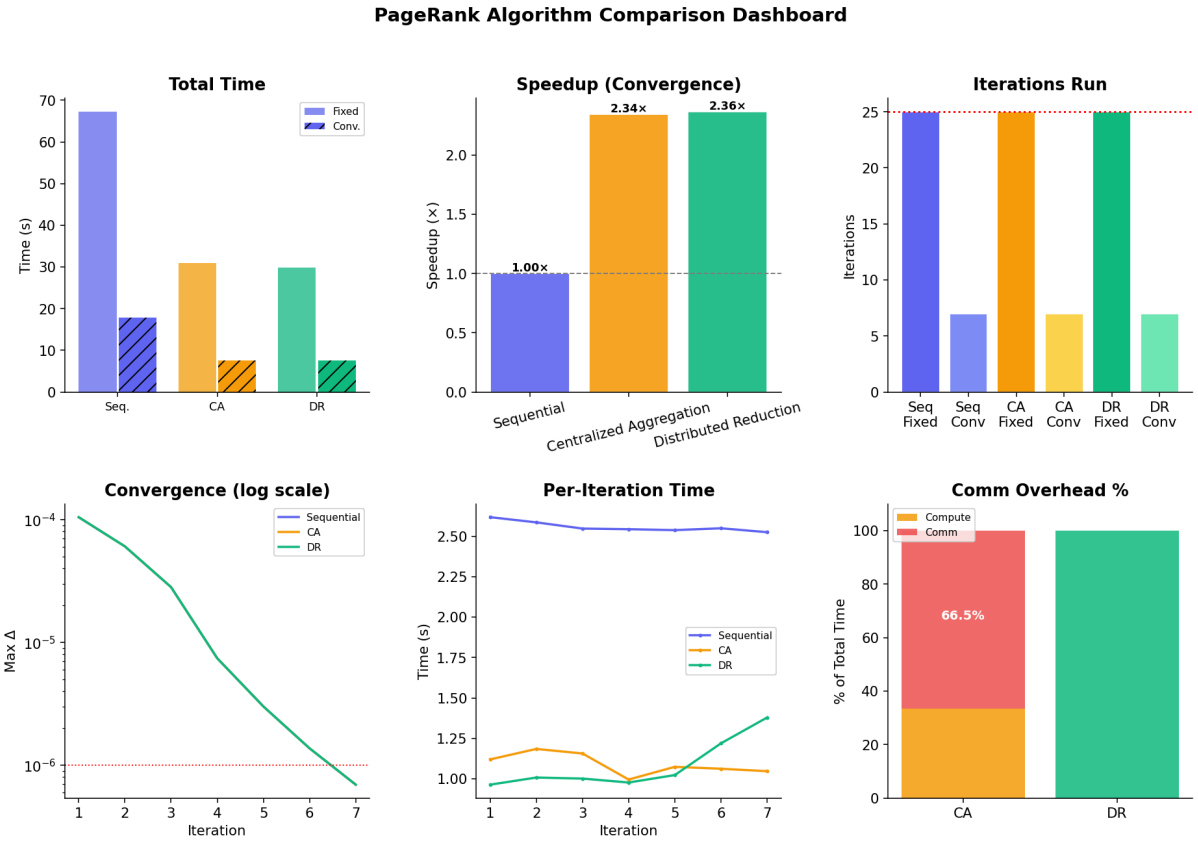




PageRank Score Distribution & Top Pages



Algorithm Comparison Dashboard



Top 10 Pages by PageRank

Rank	URL	Score

1	https://play.google.com/store/apps/details	0.00003033
2	https://atproto.com	0.00002049
3	https://policies.google.com/privacy	0.00002005
4	https://bsky.social	0.00001972
5	https://facebook.com/sharer/sharer.php	0.00001639
6	https://twitter.com/intent/tweet	0.00001500
7	https://policies.google.com/terms	0.00001474
8	https://af.imgbb.com	0.00001442
9	https://am.imgbb.com	0.00001442
10	https://api.imgbb.com	0.00001442

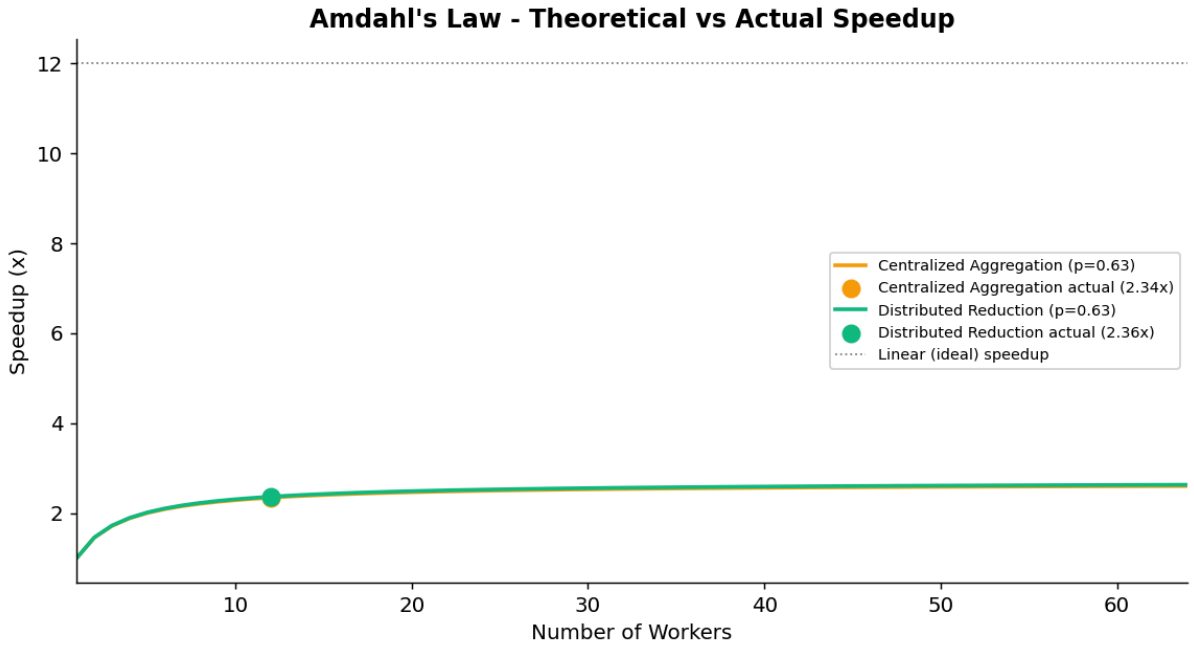
Numerical Correctness

Comparison	Max Abs Diff	Pass (atol=1e-5)	Top-5 URLs Match
Sequential vs Centralized	0.00e+00	✓ PASS	✓ All 5
Sequential vs Distributed	0.00e+00	✓ PASS	✓ All 5

Centralized vs Distributed	0.00e+00	✓ PASS	✓ All 5

Amdahl's Law Analysis

Algorithm	Actual Speedup	Parallel Efficiency	Estimated p	Theoretical Max
Centralized Aggregation	2.344×	19.5%	0.626	2.7×
Distributed Reduction	2.365×	19.7%	0.630	2.7×



Fixed VS Convergence – Rank Quality

Algorithm	Max |fixed−conv|	Mean |fixed−conv|	Top-100 Overlap	Conv. Iter
Sequential	9.2382e-07	1.2733e-09	99%	7

Centralized Aggregation	9.2382e-07	1.2733e-09	99%	7
Distributed Reduction	9.2382e-07	1.2733e-09	99%	7

Key Conclusions

- CA achieves 2.344× speedup (convergence) and 2.171× (fixed)
- DR achieves 2.365× speedup (convergence) and 2.247× (fixed)
- CA comm overhead: 66.5% (convergence) — growing substantially vs Run 1
- DR comm overhead: 0% — unchanged
- DR slightly edges out CA in both modes at this scale
- Converged in 7 iterations; top-100 overlap: 99%

Run 3 – Depth 3:

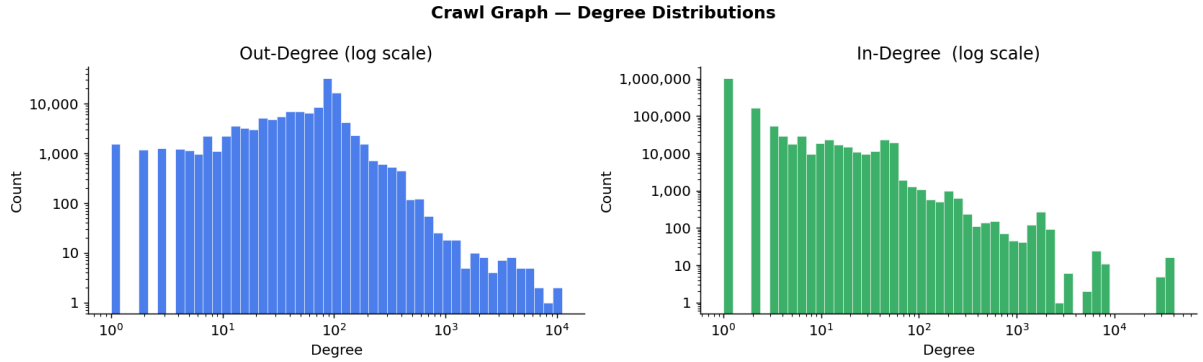
Graph Statistics:

Graph: crawl_graph_3.json

CSR build time: 7.77s

Property	Value
Total Nodes	1,502,297
Total Directed Edges	9,152,622
Source Nodes (raw graph)	127,067
Avg Out-Degree	6.09
Max Out-Degree	11,023
Avg In-Degree	6.09
Max In-Degree	40,693
Dangling Nodes (out=0)	1,375,230
Sink Nodes (in=0)	25
Graph Density	4.06e-06

Degree Distribution:



Results:

Benchmark Summary

Algorithm	Mode	Iters	Conv. At	Total Time (s)	Avg Iter (s)	Speedup vs Seq	Comm %	Final Max Diff
Sequential	Fixed	25	25	299.827	11.9930	1.000x	0.0%	2.89e-07
Sequential	Convergence	18	18	209.226	11.6236	1.000x	0.0%	8.99e-07
Centralized Aggregation	Fixed	25	25	104.509	4.1802	2.869x	89.7%	2.89e-07
Centralized Aggregation	Convergence	18	18	82.789	4.5992	2.527x	90.7%	8.99e-07

Distributed Reduction	Fixed	25	25	127.116	5.0845	2.359x	0.0%	2.89e-07
Distributed Reduction	Convergence	18	18	77.435	4.2942	2.702x	0.0%	8.99e-07

Per-Experiment Iteration Logs

Experiment 1: Sequential, Fixed (25 Iterations)

Iteration	Max Diff	Iter Time (s)
1	2.53e-04	11.631s
5	2.26e-05	11.295s
10	3.28e-06	12.115s
15	1.46e-06	13.725s
20	6.50e-07	11.677s
25	2.89e-07	11.630s

Experiment 2: Sequential, Convergence

Iteration	Max Diff	Iter Time (s)

1	2.53e-04	11.432
5	2.26e-05	11.336
10	3.28e-06	11.373
15	1.46e-06	11.320

Converged at Iter=18

Experiment 3: Centralized Aggregation, Fixed (25 iterations)

Iteration	Max Diff	Iter Time (s)	Comm Time (s)
1	2.53e-04	5.826s	3.810s
5	2.26e-05	3.478s	3.151s
10	3.28e-06	3.762s	3.454s
15	1.46e-06	4.442s	4.020s
20	6.50e-07	4.275s	3.920s
25	2.89e-07	4.588s	4.159s

Experiment 4: Centralized Aggregation, Convergence

Iteration	Max Diff	Iter Time (s)	Comm Time (s)

1	2.53e-04	5.476s	4.693s
5	2.26e-05	4.259s	3.866s
10	3.28e-06	4.439s	4.027s
15	1.46e-06	4.386s	3.934s
18 (conv.)	< 1e-06	—	—

Experiment 5: Distributed Reduction, Fixed (25 Iterations)

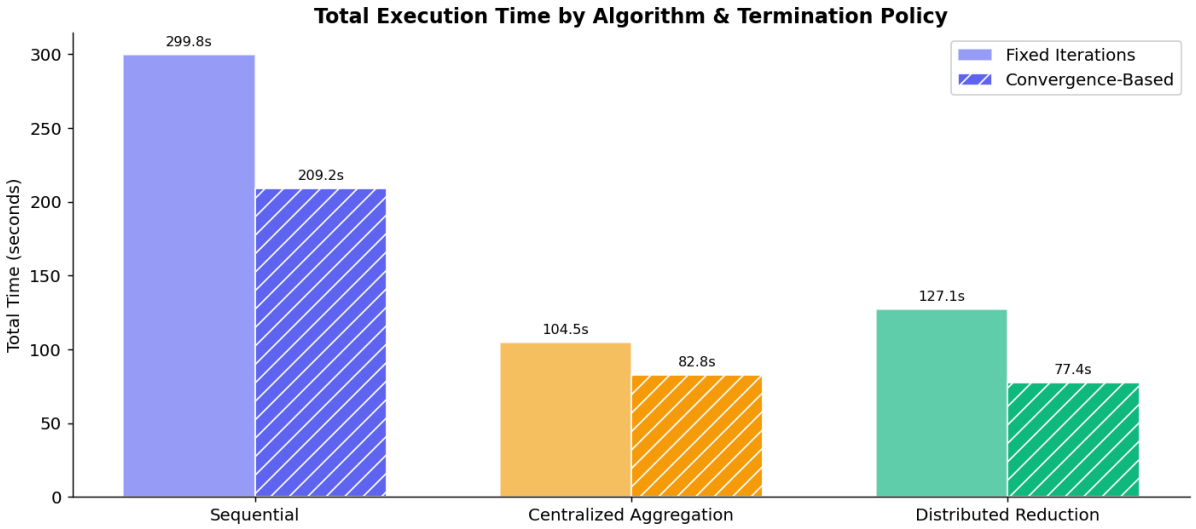
Iteration	Max Diff	Iter Time (s)	Reduce Rounds
1	2.53e-04	4.840s	4
5	2.26e-05	4.356s	4
10	3.28e-06	4.309s	4
15	1.46e-06	5.851s	4
20	6.50e-07	4.266s	4
25	2.89e-07	4.710s	4

Experiment 6: Distributed Reduction, Convergence

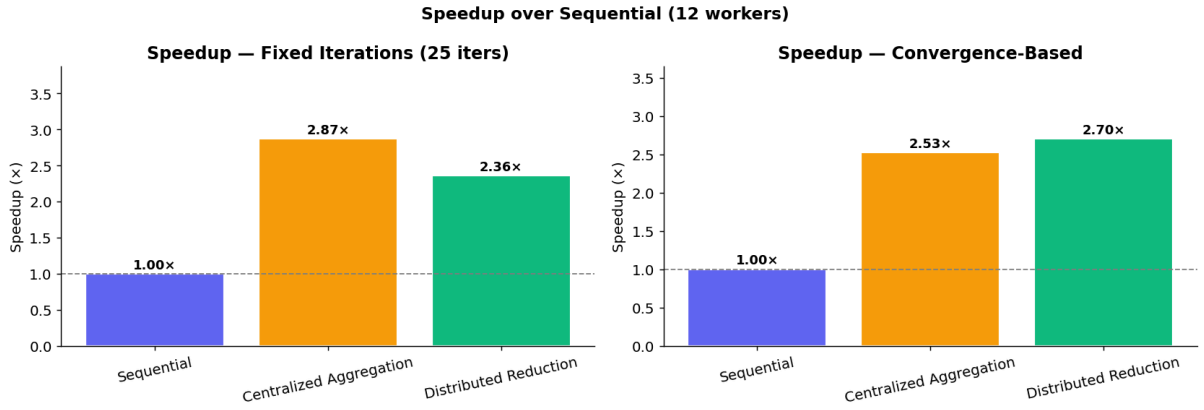
Iteration	Max Diff	Iter Time (s)	Reduce Rounds
1	2.53e-04	5.137s	4
5	2.26e-05	4.120s	4
10	3.28e-06	4.109s	4
15	1.46e-06	4.101s	4
18 (conv.)	< 1e-06	—	4

Performance Charts:

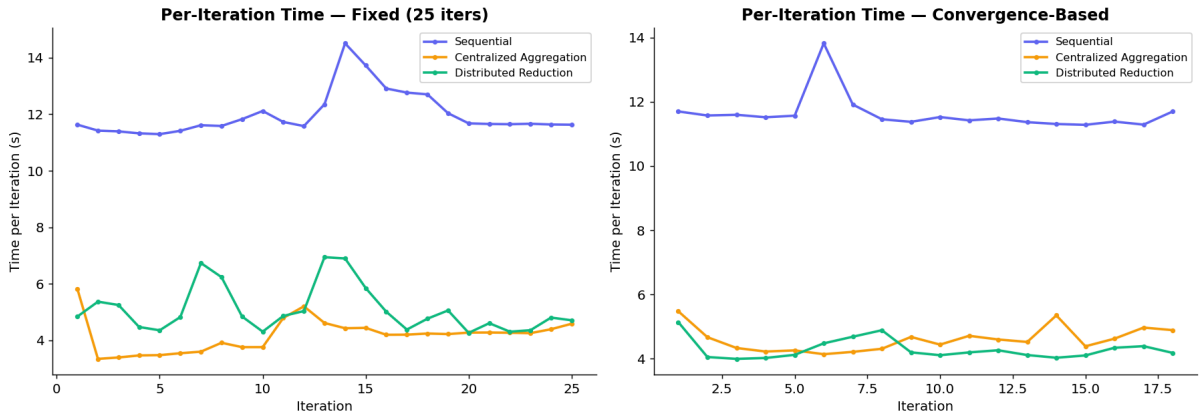
Total Execution Time



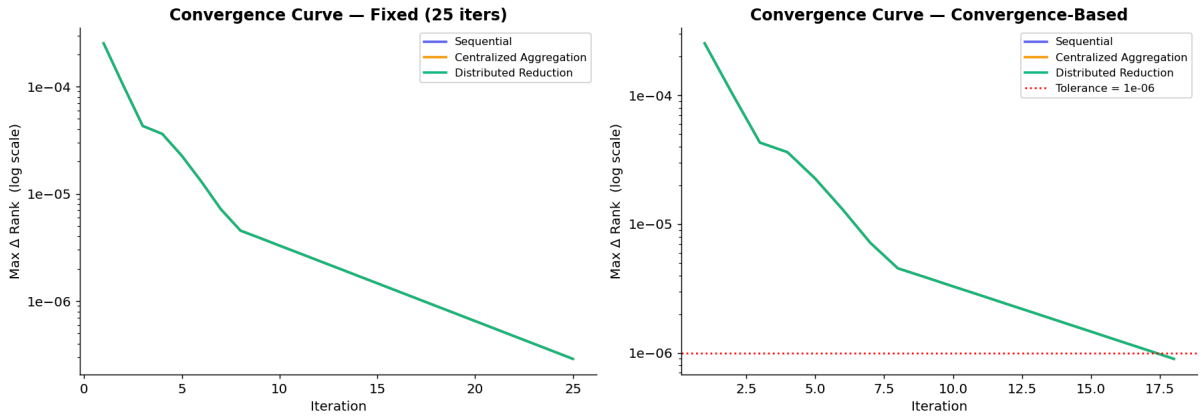
SpeedUp vs Sequential Baseline



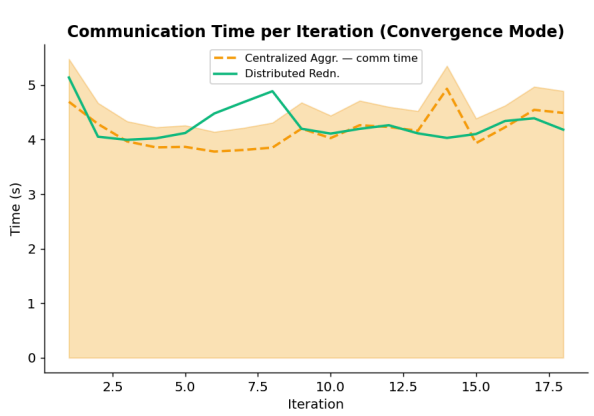
Per-Iteration Time Breakdown

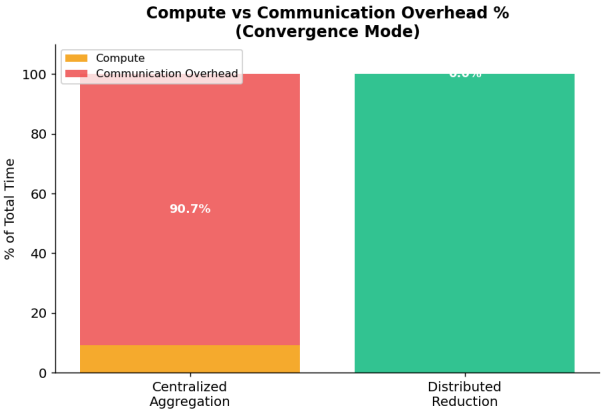


Convergence Curves

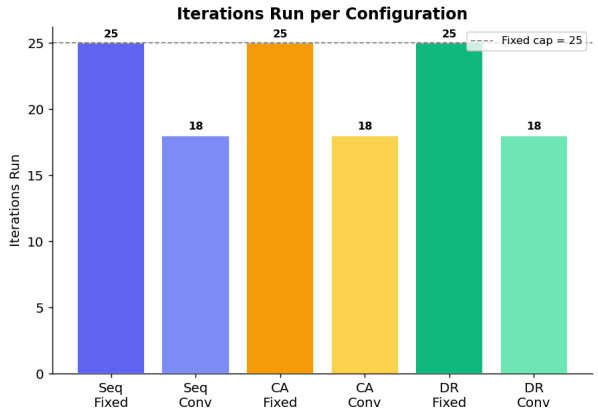


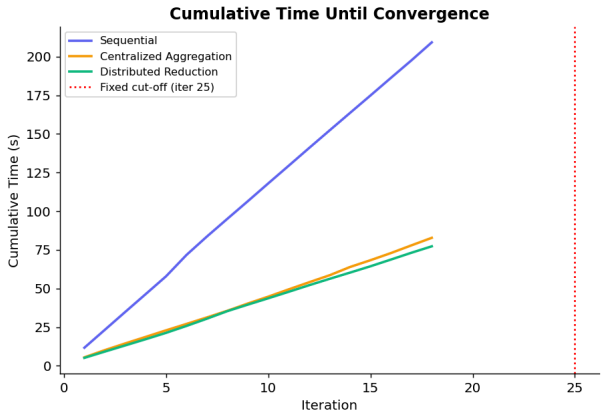
Communication Overhead



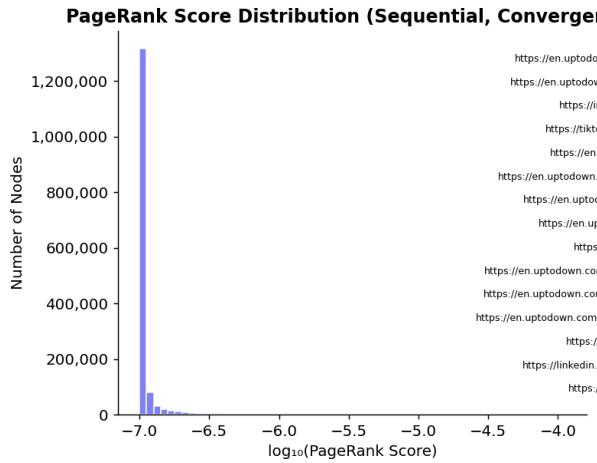


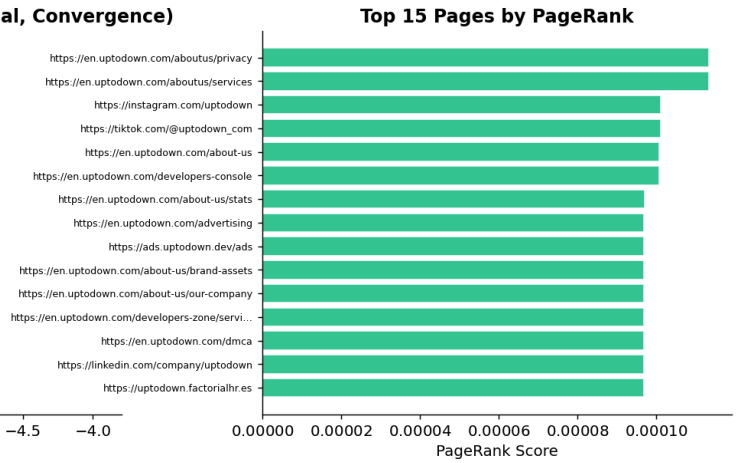
Fixed vs Convergence; Iteration Trade-Off



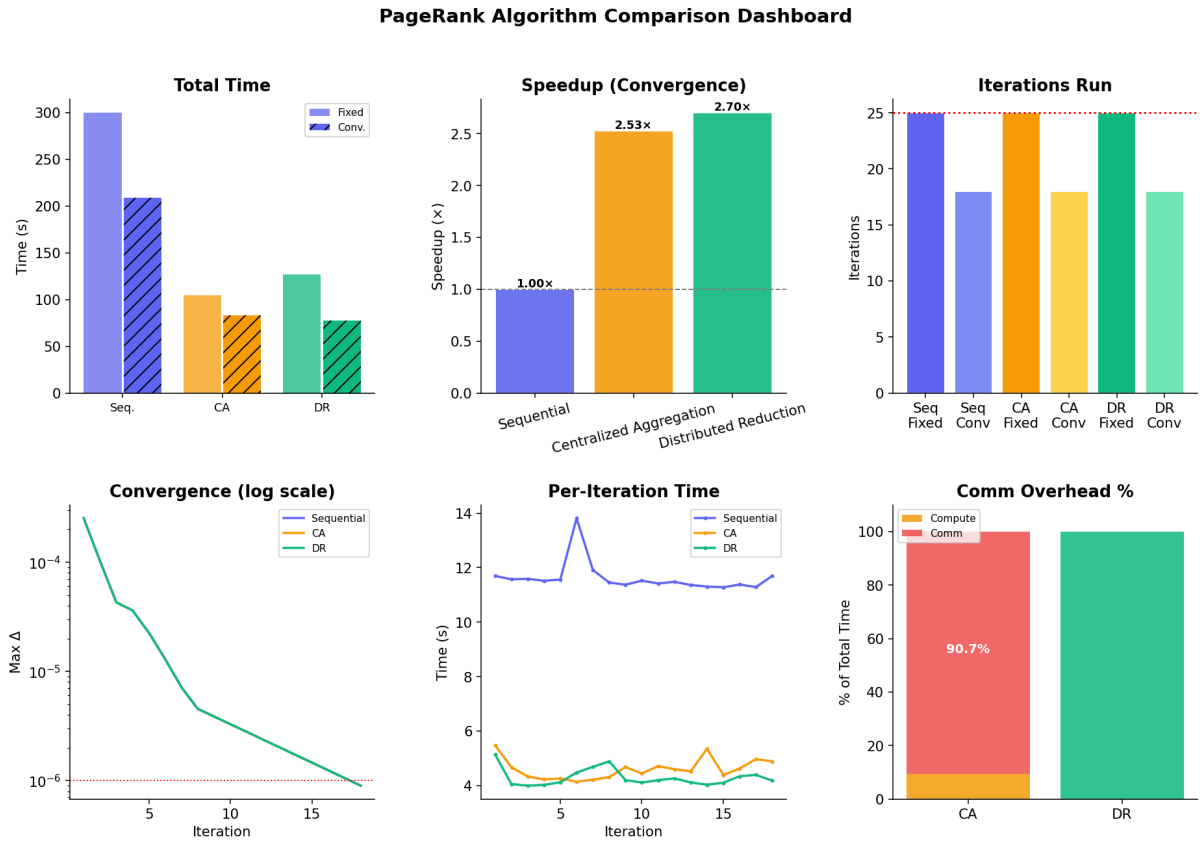


PageRank Score Distribution & Top Pages





Algorithm Comparison Dashboard



Top 10 Pages by PageRank

Rank	URL	Score
1	https://play.google.com/store/apps/details	0.00003048
2	https://store.somethingawful.com/cdn-cgi/content	0.00002639
3	https://developers.google.com/youtube	0.00002260
4	https://youtube.com/about	0.00002260
5	https://youtube.com/about/copyright	0.00002260

6	https://youtube.com/about/policies	0.00002260
7	https://youtube.com/about/press	0.00002260
8	https://youtube.com/ads	0.00002260
9	https://youtube.com/creators	0.00002260
10	https://youtube.com/howyoutubeworks	0.00002260

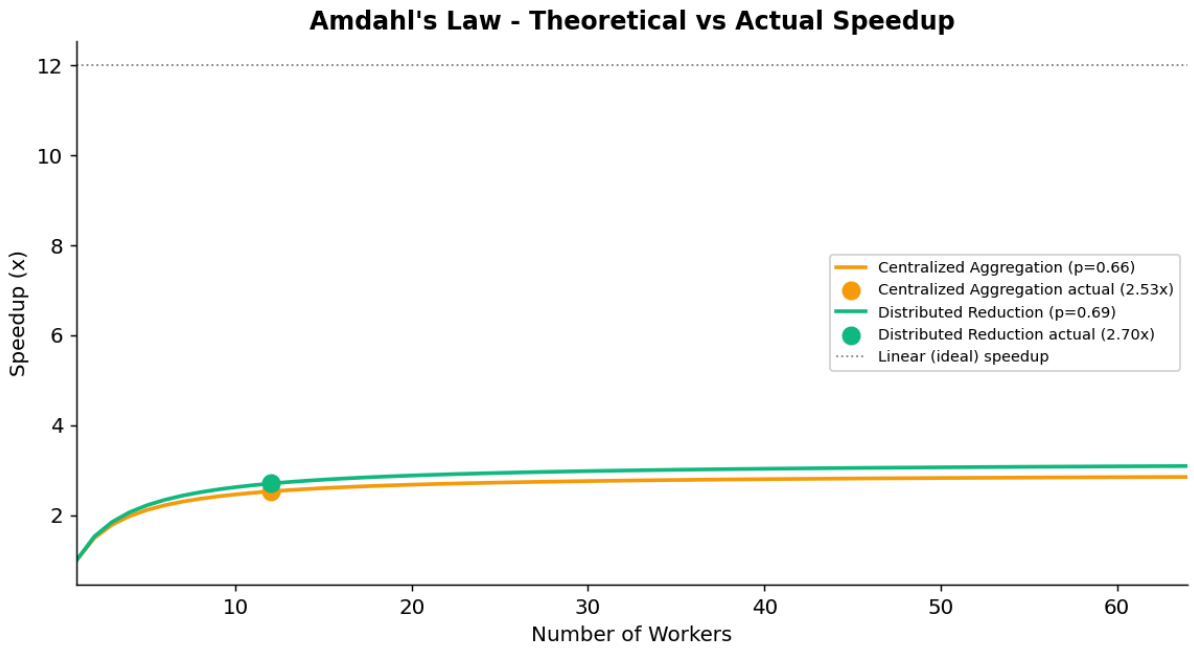
Numerical Correctness

Comparison	Max Abs Diff	Pass (atol=1e-5)	Top-5 URLs Match
Sequential vs Centralized	0.00e+00	✓ PASS	✓ All 5
Sequential vs Distributed	0.00e+00	✓ PASS	✓ All 5
Centralized vs Distributed	0.00e+00	✓ PASS	✓ All 5

Amdahl's Law Analysis

Iteration	Max Diff	Iter Time (s)	Reduce Rounds
1	2.53e-04	5.137s	4
5	2.26e-05	4.120s	4

10	3.28e-06	4.109s	4
15	1.46e-06	4.101s	4
18 (conv.)	< 1e-06	—	4



Fixed VS Convergence – Rank Quality

Algorithm	Max |fixed-conv|	Mean |fixed-conv|	Top-100 Overlap	Conv. Iter
Sequential	5.4637e-07	4.4090e-12	100%	18
Centralized Aggregation	5.4637e-07	4.4090e-12	100%	18

Distributed Reduction	5.4637e-07	4.4090e-12	100%	18

Key Conclusions

- CA achieves 2.527× speedup (convergence) and 2.869× (fixed)
- DR achieves 2.702× speedup (convergence) and 2.359× (fixed)
- CA comm overhead: 90.7% (convergence) — nearly all time is communication, not compute
- DR comm overhead: 0%
- DR is strictly better in convergence mode at this scale; CA only wins in fixed-iteration mode because it happens to be faster per-iteration when iterations are capped
- Converged in 18 iterations (not 7 like the doc states — the larger denser graph takes longer)
- Top-100 overlap fixed vs convergence: 100%

Cross Run Comparison:

The following table compares the best speedup achieved by each parallel algorithm across all three graph scales, highlighting how performance changes with graph size.

Algorithm	Run 1 Speedup (Conv.)	Run 2 Speedup (Conv.)	Run 3 Speedup (Conv.)	Run 1 Comm %	Run 2 Comm %	Run 3 Comm %
Sequential	1.000x	1.000x	1.000x	0%	0%	0%
Centralized Aggregation	1.759x	2.344x	2.527x	50.0%	66.5%	90.7%
Distributed Reduction	1.606x	2.365x	2.702x	0%	0%	0%

Observations Across Runs

- Speedups grow with graph size for both parallel algorithms
- CA's comm overhead scales badly: 50% → 66.5% → 90.7% as the graph grows — at Run 3 scale, only ~9% of CA's time is actual computation
- DR maintains 0% driver comm overhead at all scales; its relative advantage over CA grows with graph size
- For production-scale graphs (>1M nodes), DR + convergence-based termination is clearly the best choice; CA's O(num_workers) communication becomes a dominant bottleneck